



中國農業大學
China Agricultural University

人工智能——深度强化学习与控制

胡标



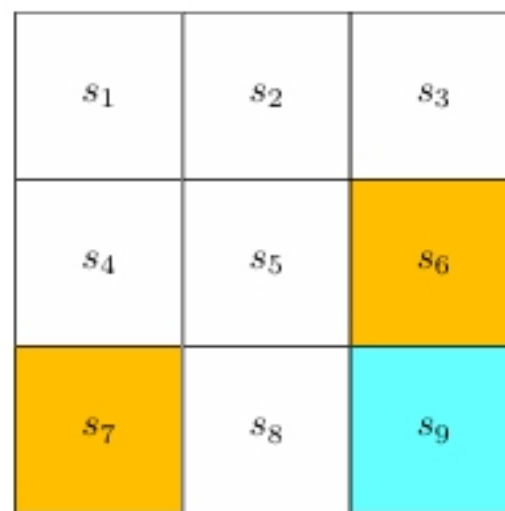
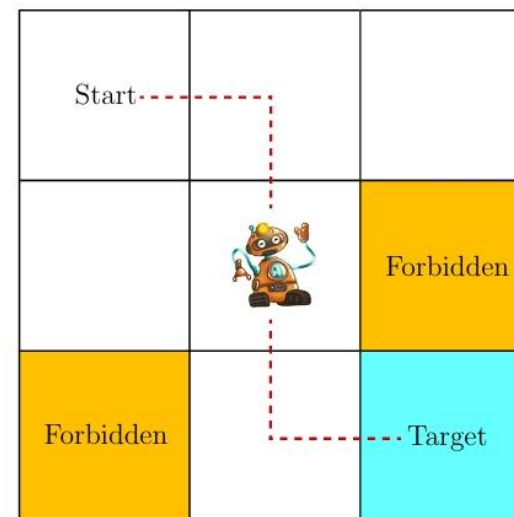
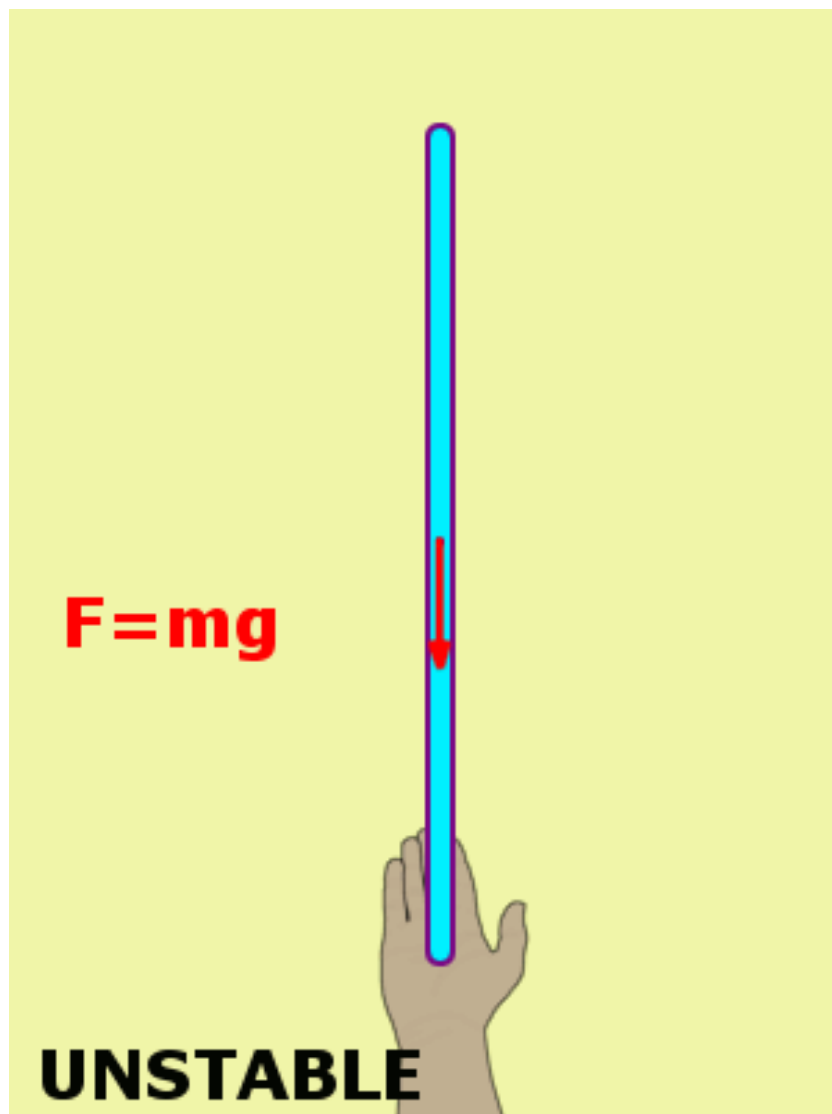


目录

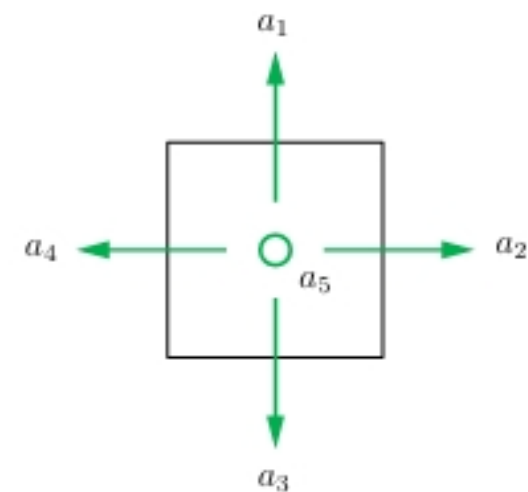
Contents

1. 强化学习基本要素
2. 深度Q神经网络算法DQN
3. 深度确定性策略梯度算法DDPG
4. 近端策略优化算法PPO
5. 领域随机化
6. 实际环境搭建

一个例子



(a) States



(b) Actions

□ 智能体 (Agent)

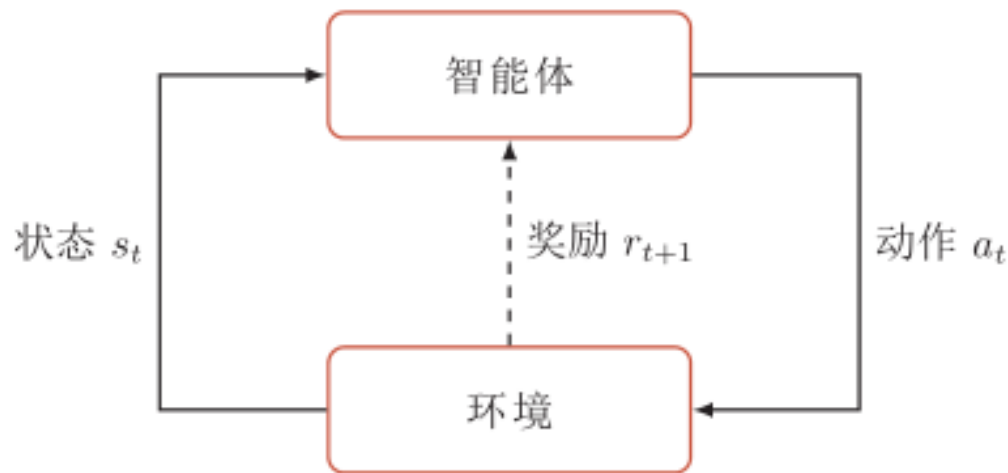
□ 感知外界环境的状态 (State) 和奖励反馈 (Reward) , 并进行学习和决策。

智能体的决策功能是指根据外界环境的状态来做出不同的动作 (Action) , 而学习功能是指根据外界环境的奖励来调整策略。

□ 环境 (Environment)

□ 智能体外部的所有事物, 并受智能体动作的影响而改变其状态, 并反馈给智能体相应的奖励。

- 强化学习问题可以描述为一个智能体从与环境的交互中不断学习以完成特定目标（比如取得最大奖励值）。
- 强化学习就是智能体不断与环境进行交互，并根据经验调整其策略来最大化其长远的所有奖励的累积值。



- 环境的**状态集合**： S ;
- 智能体的**动作集合**： A ;
- **状态转移概率**： $p(s'|s, a)$ ，即智能体根据当前状态 s 做出一个动作 a 之后，下一个时刻环境处于不同状态 s' 的概率;
- **即时奖励**： $R : S \times A \times S' \rightarrow R$ ，即智能体根据当前状态做出一个动作之后，环境会反馈给智能体一个奖励，这个奖励和动作之后下一个时刻的状态有关。

	a_1 (upward)	a_2 (rightward)	a_3 (downward)	a_4 (leftward)	a_5 (still)
s_1	s_1	s_2	s_4	s_1	s_1
s_2	s_2	s_3	s_5	s_1	s_2
s_3	s_3	s_3	s_6	s_2	s_3
s_4	s_1	s_5	s_7	s_4	s_4
s_5	s_2	s_6	s_8	s_4	s_5
s_6	s_3	s_6	s_9	s_5	s_6
s_7	s_4	s_8	s_7	s_7	s_7
s_8	s_5	s_9	s_8	s_7	s_8
s_9	s_6	s_9	s_9	s_8	s_9

比如：

$$p(s_1|s_1, a_2) = 0$$

$$p(s_2|s_1, a_2) = 1$$

$$p(s_3|s_1, a_2) = 0$$

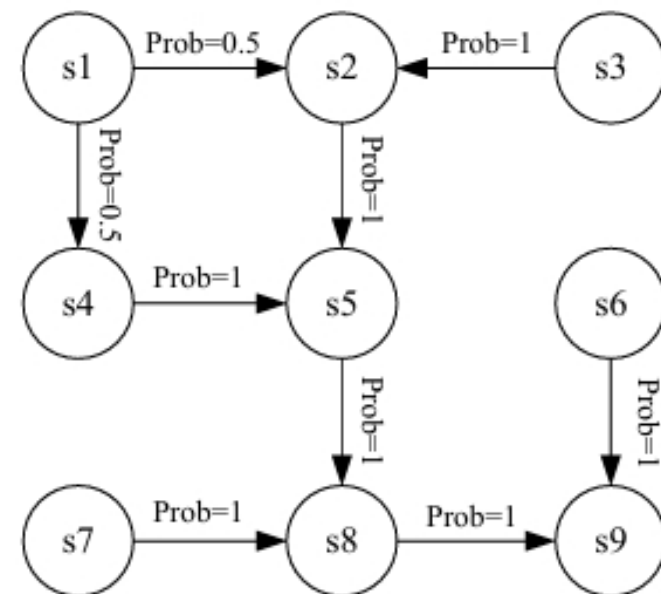
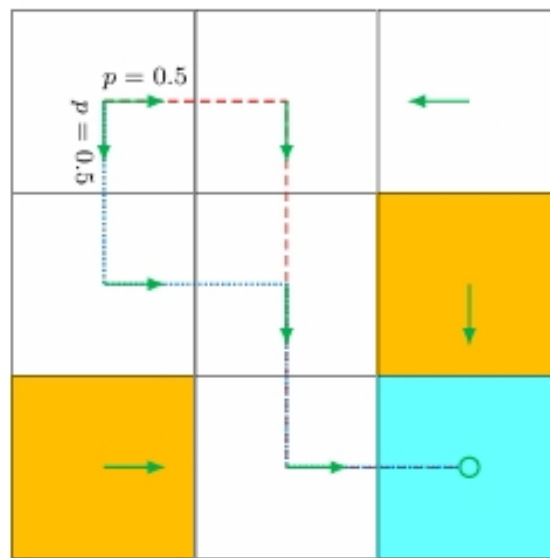
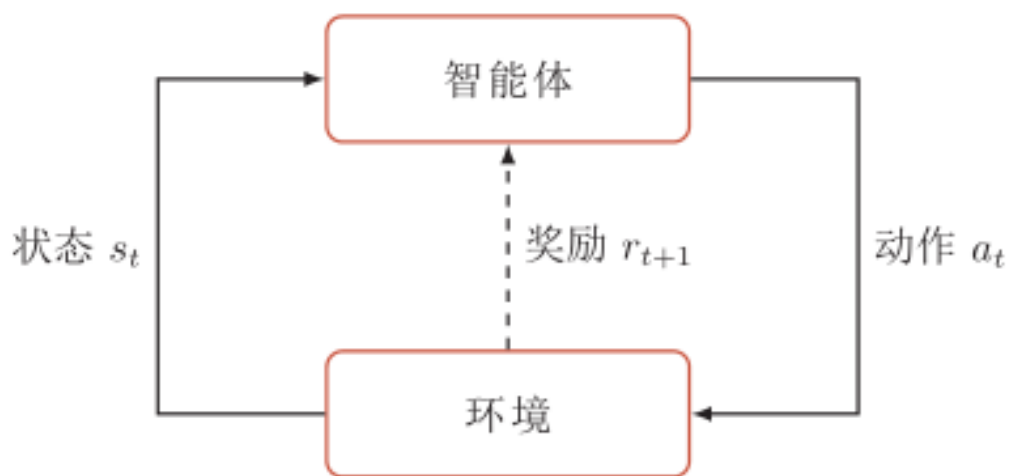
$$p(s_4|s_1, a_2) = 0$$

$$p(s_5|s_1, a_2) = 0$$

□ 马尔可夫过程

$$P(s_{t+1}|s_t, a_t, \dots, s_0, a_0) = p(s_{t+1}|s_t, a_t)$$

$$P(r_{t+1}|s_t, a_t, \dots, s_0, a_0) = p(r_{t+1}|s_t, a_t)$$



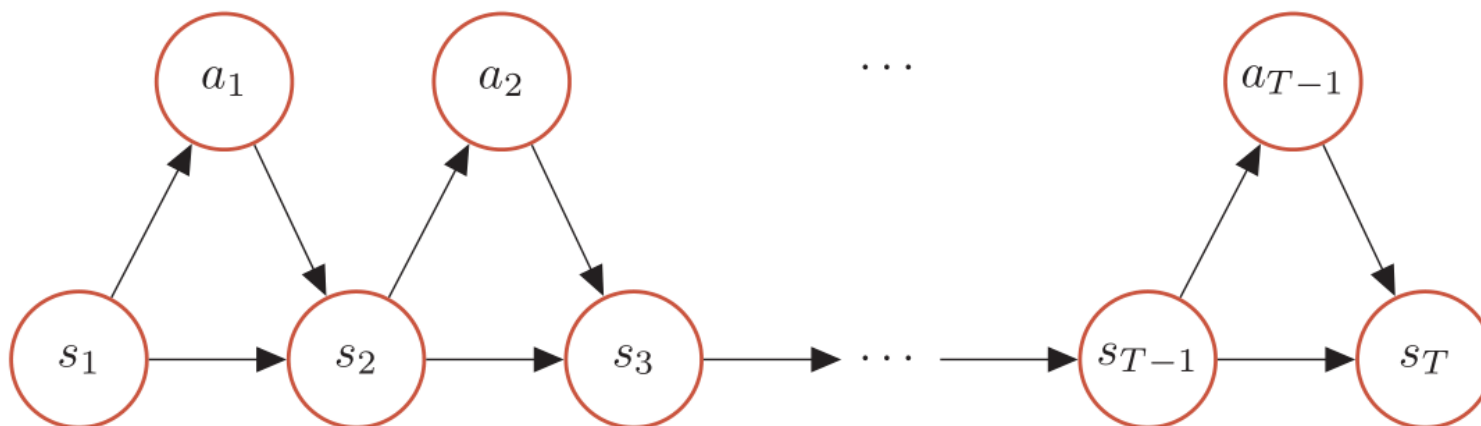
$s_0, a_0, s_1, r_1, a_1, \dots, s_{t-1}, a_{t-1}, s_t, r_t, \dots$

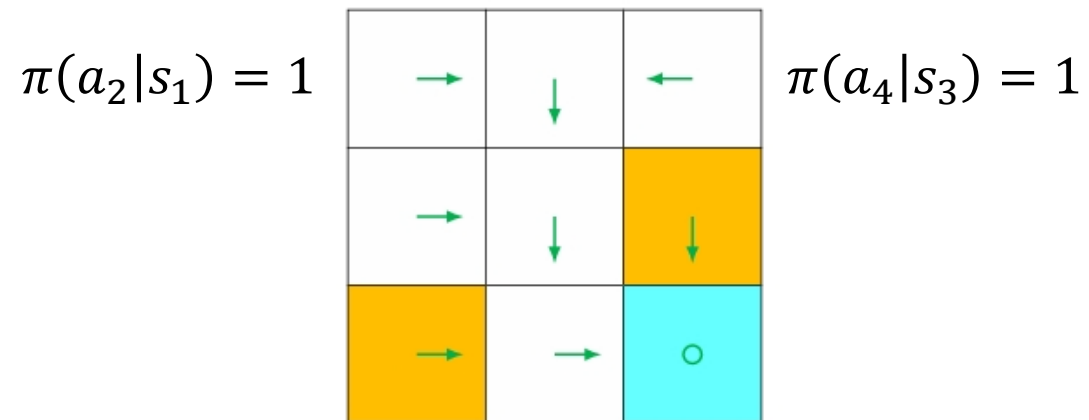
□ 马尔可夫决策过程的一个轨迹 (trajectory)

$$\tau = s_0, a_0, s_1, r_1, a_1, \dots, s_{T-1}, a_{T-1}, s_T$$

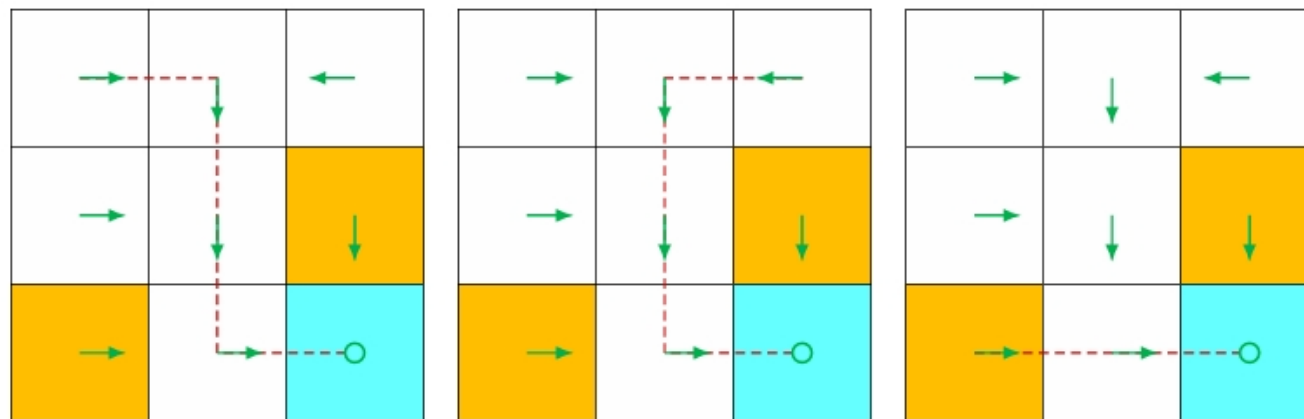
□ τ 的概率

$$p(\tau) = p(s_0, a_0, s_1, r_1, a_1, \dots, s_{T-1}, a_{T-1}, s_T) = p(s_0) \prod_{t=0}^{T-1} \pi(a_t | s_t) p(s_{t+1} | s_t, a_t)$$





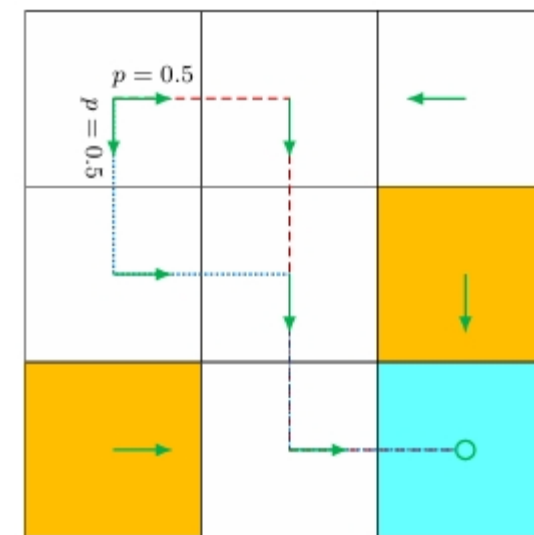
(a) A deterministic policy



(b) Trajectories obtained from the policy

确定性策略

$\pi(a_2|s_1) = 0.5$
 $\pi(a_3|s_1) = 0.5$



随机策略

- 给定策略 $\pi(a|s)$ ，智能体和环境一次交互过程的轨迹 τ 所收到的累积奖励为总回报 (return)

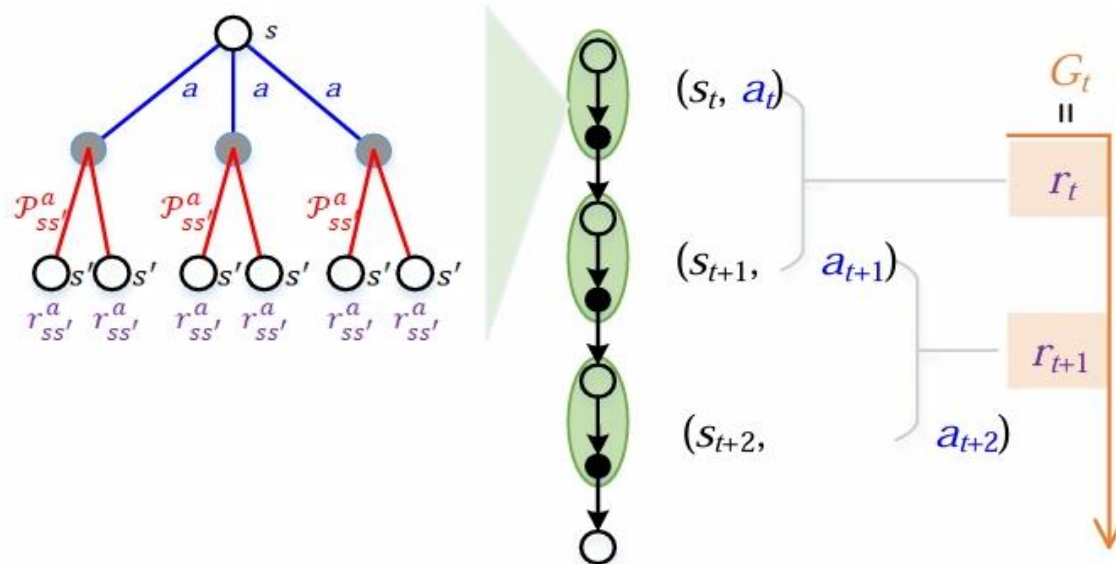
$$G(\tau) = \sum_{t=0}^{T-1} \gamma^t r_{t+1}$$

- $\gamma \in [0,1]$ 是折扣率。当 γ 接近于0时，智能体更在意短期回报；而当 γ 接近于1时，长期回报变得更重要。
- 环境中有一个或多个特殊的终止状态 (terminal state)

□ 强化学习的目标是学习到一个策略 $\pi_{\theta}(a|s)$ 来**最大化期望回报** (expected return)

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)}[G(\tau)] = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=0}^{T-1} \gamma^t r_{t+1} \right]$$

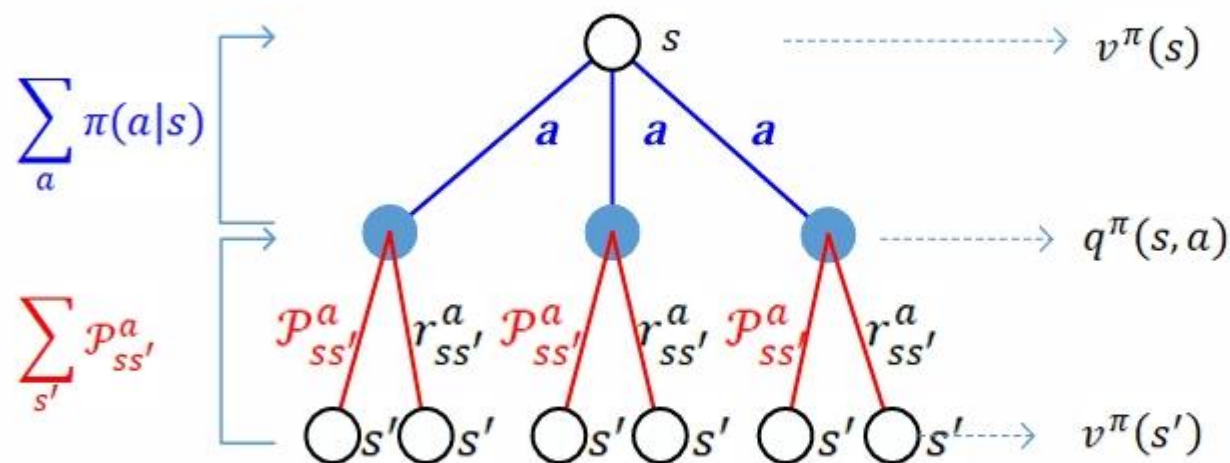
□ θ 为策略函数的参数



如何评估策略 $\pi_\theta(a|s)$



$$v^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) q^\pi(s, a)$$



$$q^\pi(s, a) = \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a (r_{ss'}^a + \gamma v^\pi(s'))$$

□ 状态值函数：从状态 s 开始，执行策略 π 得到的期望总回报

$$V^{\pi}(s) = E_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=0}^{T-1} \gamma^t r_{t+1} \mid \tau_{s_0} = s \right] = E_{\tau \sim p_{\theta}(\tau)} \left[r_1 + \sum_{t=1}^{T-1} \gamma^{t-1} r_{t+1} \mid \tau_{s_0} = s \right]$$

$$= E_{\tau \sim p_{\theta}(\tau)} E_{s' \sim p(s'|s,a)} \left[r(s, a, s') + \gamma \sum_{t=1}^{T-1} \gamma^{t-1} r_{t+1} \mid \tau_{s_1} = s' \right]$$

$$= E_{\tau \sim p_{\theta}(\tau)} E_{s' \sim p(s'|s,a)} [r(s, a, s') + \gamma V^{\pi}(s')] \leftarrow \text{贝尔曼方程}$$

□ 状态-动作值函数 (Q函数)

$$\begin{aligned} Q^\pi(s, a) &= \mathbb{E}_{s' \sim p_\theta(s'|s, a)} [r(s, a, s') + \gamma V^\pi(s')] \\ &= \mathbb{E}_{s' \sim p_\theta(s'|s, a)} \left[r(s, a, s') + \gamma \mathbb{E}_{a' \sim \pi(a'|s')} [Q^\pi(s', a')] \right] \leftarrow \text{贝尔曼方程} \end{aligned}$$

□ 策略学习过程

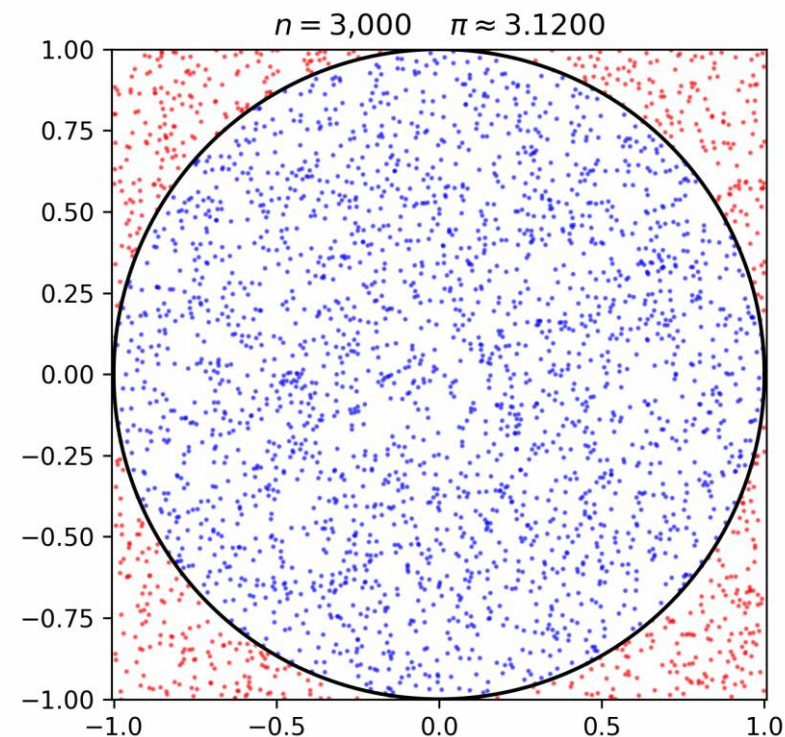
□ 通过采样的方式来计算值函数

$$Q_{\pi}(s, a) \approx \hat{Q}_{\pi}(s, a) = \frac{1}{N} \sum_{n=1}^N G(\tau^{(n)})$$

□ 当 $N \rightarrow \infty$ 时, $\hat{Q}_{\pi}(s, a) \rightarrow Q_{\pi}(s, a)$ 。

□ 在估计出 $Q_{\pi}(s, a)$ 之后, 就可以进行策略改进。

□ 然后在新的策略下重新通过采样来估计Q函数, 并不断重复, 直至收敛。





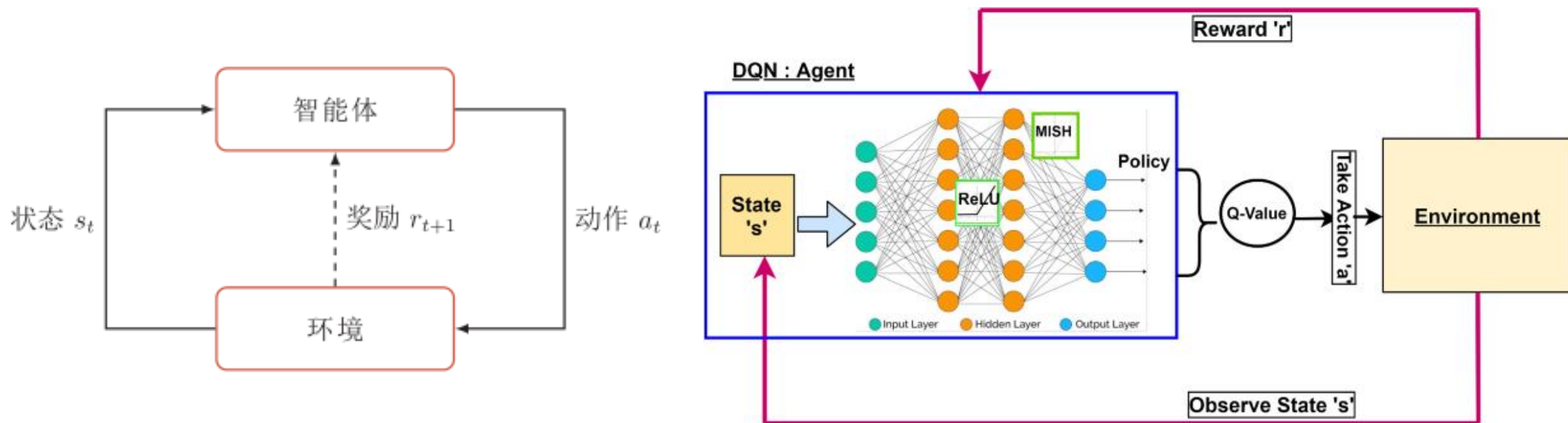
目录

Contents

1. 强化学习基本要素
2. 深度Q神经网络算法DQN
3. 深度确定性策略梯度算法DDPG
4. 近端策略优化算法PPO
5. 领域随机化
6. 实际环境搭建

- 为了在连续的状态和动作空间中计算值函数 $Q_{\pi}(s, a)$ ，我们可以用一个函数 $Q_{\theta}(s, a)$ 来表示近似计算，称为值函数近似 (Value Function Approximation)

$$Q_{\theta}(s, a) \approx Q^{\pi}(s, a)$$



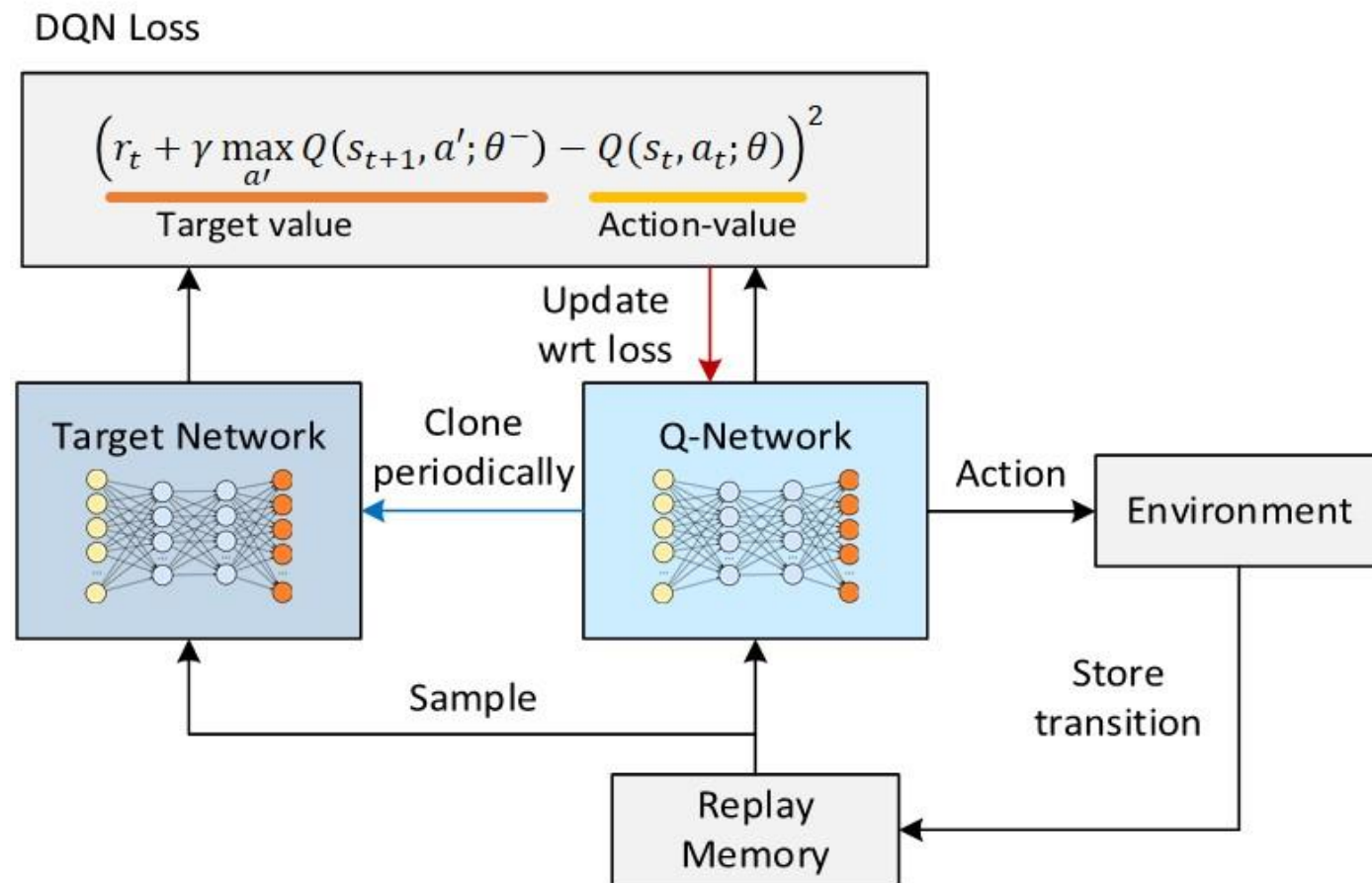
$$\mathcal{L}(s, a, s'; \theta) = \left(r + \gamma \max_{a'} (Q_{\theta}(s', a')) - Q_{\theta}(s, a) \right)^2$$

□ 存在两个问题：

- 目标不稳定，参数学习的目标依赖于参数本身；
- 样本之间有很强的相关性。

深度Q网络

- 一是目标网络冻结 (freezing target networks), 即在一个时间段内固定目标中的参数, 来稳定学习目标;
- 二是经验回放 (experience replay), 构建一个经验池来去除数据相关性。



算法 15.5: 带经验回放的深度 Q 网络

输入: 状态空间 $\mathcal{S}$, 动作空间 $\mathcal{A}$, 折扣率 γ , 学习率 α

```

1 初始化经验池  $\mathcal{D}$ , 容量为  $N$ ;
2 随机初始化  $Q$  网络的参数  $\phi$ ;
3 随机初始化目标  $Q$  网络的参数  $\hat{\phi} = \phi$ ;
4 repeat
5     初始化起始状态  $s$ ;
6     repeat
7         在状态  $s$ , 选择动作  $a = \pi^\epsilon$ ;
8         执行动作  $a$ , 观测环境, 得到即时奖励  $r$  和新的状态  $s'$ ;
9         将  $s, a, r, s'$  放入  $\mathcal{D}$  中;
10        从  $\mathcal{D}$  中采样  $ss, aa, rr, ss'$ ;
11        
$$y = \begin{cases} rr, & ss' \text{ 为终止状态,} \\ rr + \gamma \max_{a'} Q_{\hat{\phi}}(ss', a'), & \text{否则} \end{cases};$$

12        以  $(y - Q_{\phi}(s, a))^2$  为损失函数来训练  $Q$  网络;
13         $s \leftarrow s'$ ;
14        每隔  $C$  步,  $\hat{\phi} \leftarrow \phi$ ;
15    until  $s$  为终止状态;
16 until  $\forall s, a, Q_{\phi}(s, a)$  收敛;
    输出:  $Q$  网络  $Q_{\phi}(s, a)$ 

```

□ **经验回放机制**: 算法使用经验池 $\mathcal{D}$ 来存储智能体与环境交互的经验样本 (s, a, r, s') 。这些样本被随机抽样, 用于训练深度 Q 网络 Q_{ϕ} 。这种方法减少了样本间的相关性, 提升了训练的稳定性和效率。

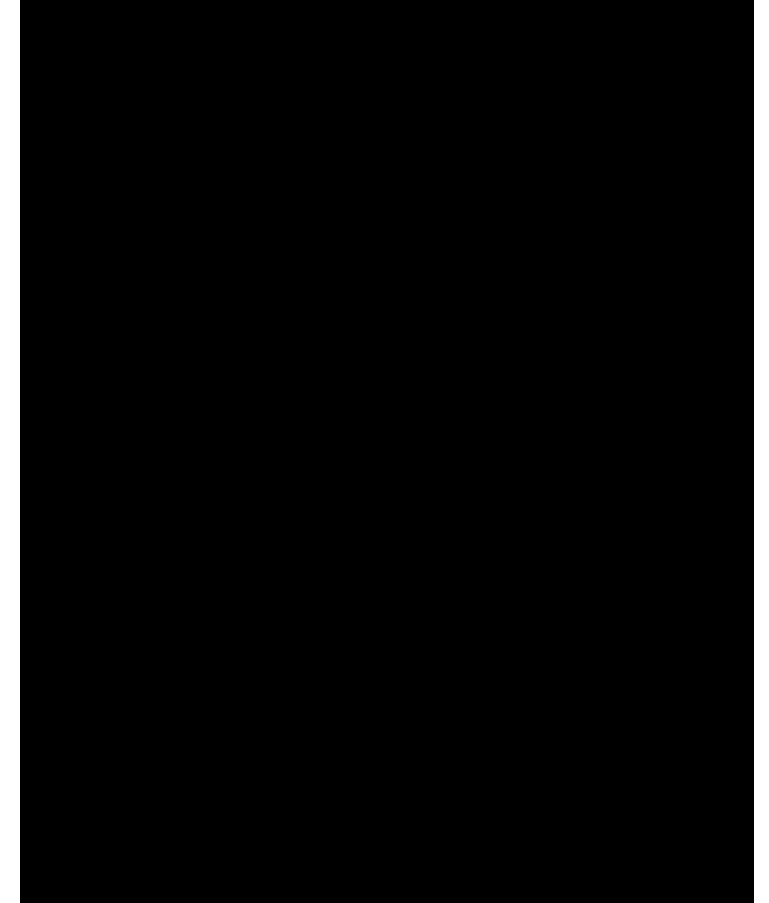
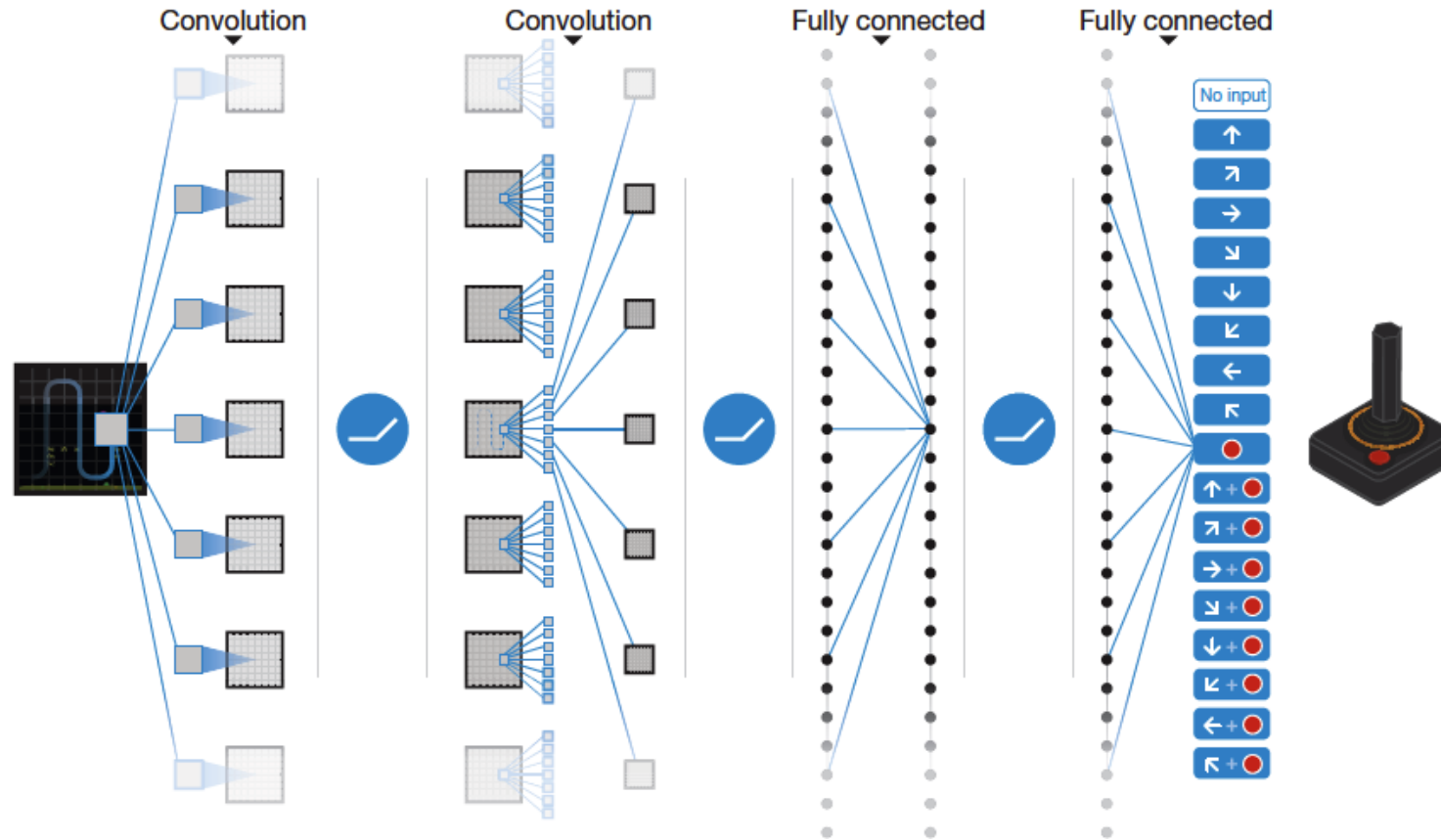
□ **目标网络与主网络的分离**: 算法引入了目标 Q 网络 $Q_{\hat{\phi}}$, 用于计算目标值 y 。目标网络的参数 $\hat{\phi}$ 定期从主网络的参数 ϕ 同步而来, 但在训练过程中保持不变。这种分离有助于避免目标值频繁更新引起的不稳定。

□ **ϵ -贪婪策略**: 在每一步中, 算法通过 ϵ -贪婪策略选择动作 a 。具体来说, 以概率 ϵ 随机选择动作 (探索), 以概率 $1 - \epsilon$ 选择当前 Q 网络输出的最佳动作 (利用)。这种策略平衡了探索新状态与利用当前策略的权衡。

DQN in Atari



中國農業大學
China Agricultural University



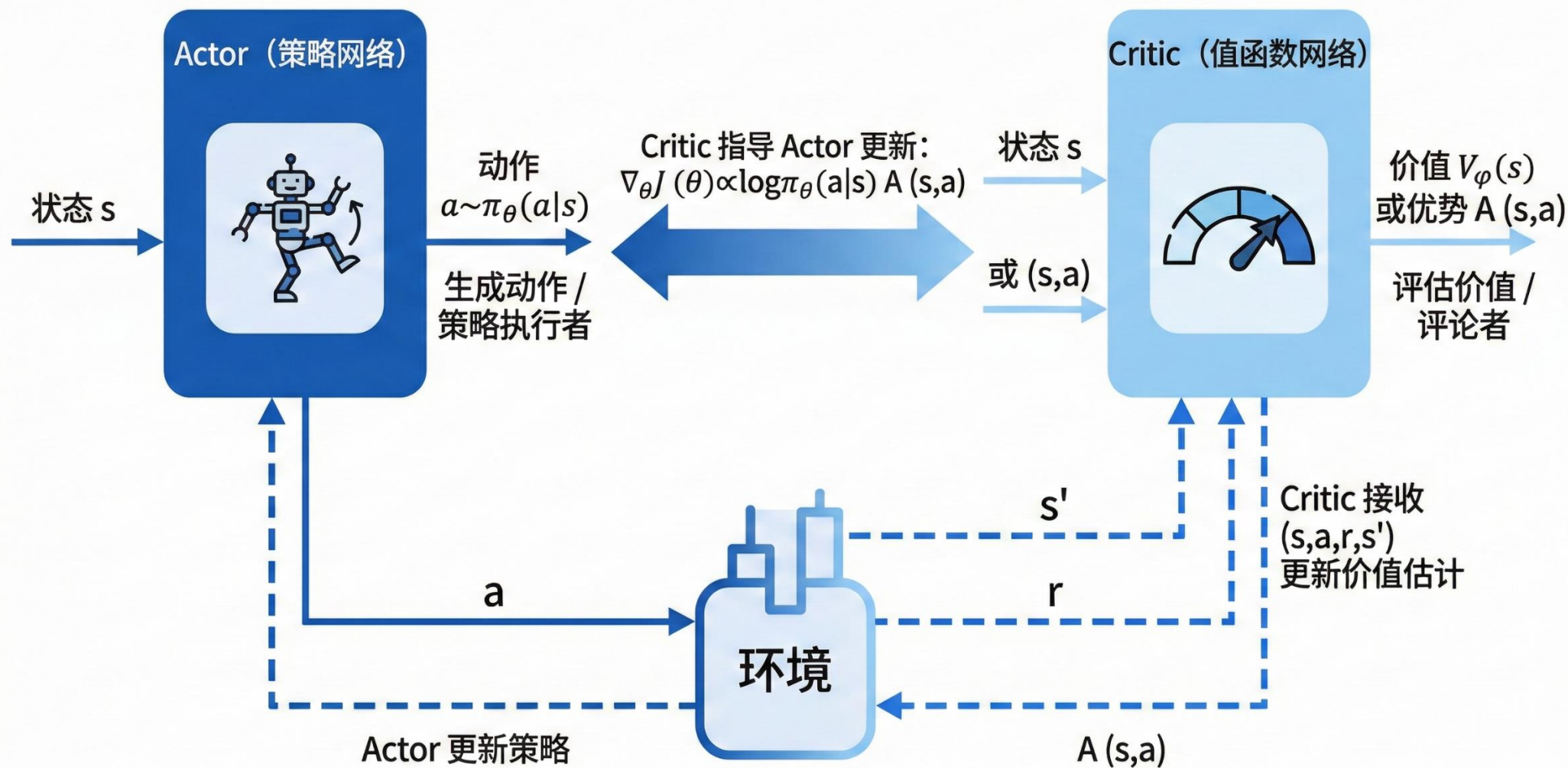
Mnih, Volodymyr, et al. "Human-level control through deep reinforcement learning." *Nature* 518.7540 (2015): 529-533.



目录

Contents

1. 强化学习基本要素
2. 深度Q神经网络算法DQN
3. 深度确定性策略梯度算法DDPG
4. 近端策略优化算法PPO
5. 领域随机化
6. 实际环境搭建



优势函数 $A^\pi(s, a)$ 定义为动作值函数与状态值函数之差：

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

它衡量了在状态 s 下执行特定动作 a 相对于遵循当前策略 π 的“平均表现”有多好或多差。

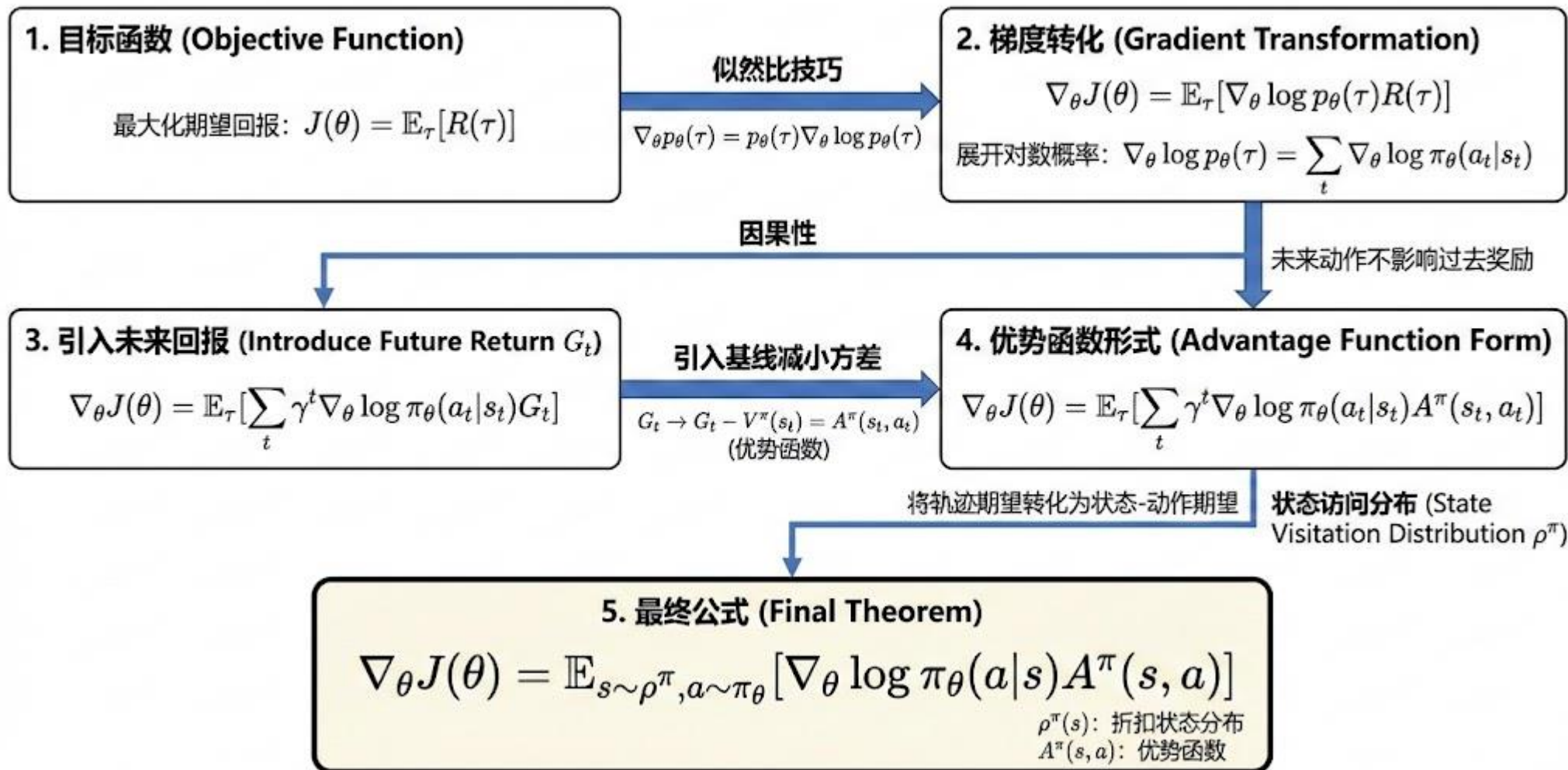
- $A^\pi(s, a) > 0$ ：表明动作 a 比平均动作更好。比如在结冰路面上，点刹比急刹的效果好，优势值为正。
- $A^\pi(s, a) < 0$ ：表明动作 a 比平均动作更差。比如在高速行驶时急打方向盘，优势值为负。
- $A^\pi(s, a) = 0$ ：表明动作 a 处于平均水平。比如在平直道路上匀速行驶。

使用优势 $A^\pi(s, a)$ 代替原始的回报 G_t ，策略梯度定理可以写为：

$$\nabla_\theta J(\theta) = \mathbb{E}_{s \sim \rho^\pi, a \sim \pi_\theta} \left[\nabla_\theta \log \pi_\theta(a | s) A^\pi(s, a) \right]$$

这个公式的直观解释是：在状态 s 下执行动作 a 的“惊喜程度”乘以这个动作的“相对质量”，如果动作比平均水平好，就增加选择它的概率；反之则减少。

策略梯度定理推导过程总结 (Summary of Policy Gradient Theorem Derivation)



1. 整体框架

Actor网络 (Policy Network) :
参数 θ , 输出 $\pi_{\theta}(a|s)$

Critic网络 (Value Network) :
参数 w , 输出 $V_w(s)$

优势函数 $A_{\pi}(s, a)$
由Critic估计

策略梯度:

$$\nabla_{\theta} J(\theta) = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a|s) A_{\pi}(s, a)]$$

2. 优势函数估计

2.1 TD误差 (单步) :

$$\delta_t = r_t + \gamma V_w(s_{t+1}) - V_w(s_t)$$

(方差小, 可能有偏差)

2.2 n步回报:

$$A^{(n)} = \sum \gamma^k r_{t+k} + \gamma^n V_w(s_{t+n}) - V_w(s_t)$$

2.3 GAE (广义优势估计) :

$$A_t^{GAE} = \sum (\gamma \lambda)^l \delta_{t+l}$$

(平衡偏差与方差)

3. 更新流程 (在线TD示例)

步骤1: 计算TD误差

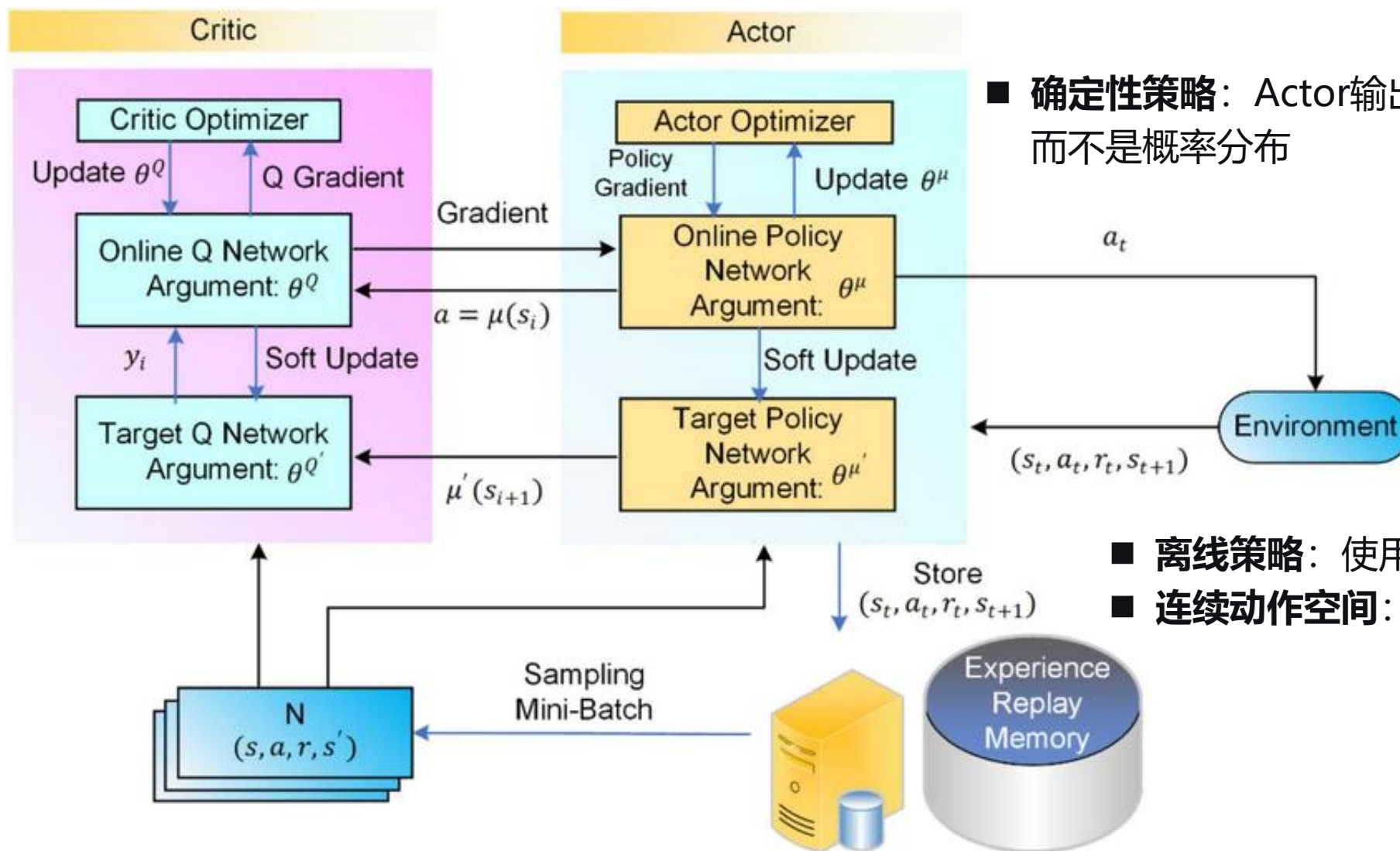
$$\delta_t = r_t + \gamma V_w(s_{t+1}) - V_w(s_t)$$

步骤2: 更新Critic (价值网络)

Loss: $L_w = \frac{1}{2} \delta_t^2$;
更新: $w \leftarrow w + \alpha_w \delta_t \nabla_w V_w(s_t)$

步骤3: 更新Actor (策略网络)

梯度: $\nabla_{\theta} J(\theta) \approx \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \cdot \delta_t$
更新: $\theta \leftarrow \theta + \alpha_{\theta} \delta_t \nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$



■ **确定性策略:** Actor输出确定性动作 $a = \mu_\theta(s_i)$, 而不是概率分布

■ **离线策略:** 使用经验回放和独立的目标网络

■ **连续动作空间:** 直接输出连续动作值

策略迭代

阶段一：收集经验

1. 选择动作: $a_t = \mu_\theta(s_t) + N_t$ (噪声 N_t)

2. 执行动作: 获得 r_t, s_{t+1}

3. 存储转移: $(s_t, a_t, r_t, s_{t+1}) \rightarrow$ 经验回放缓冲区 D

阶段二：更新网络

采样小批量 N 个转移

步骤1：更新Critic

1. 计算目标Q值

$$y_i = r_i + \gamma Q_{\phi'}(s_{t+1}, \mu_{\theta'}(s_{t+1}))$$

2. 最小化均方贝尔曼误差

$$L(\phi) = \frac{1}{N} \sum_i (y_i - Q_\phi(s_i, a_i))^2$$

3. 梯度更新Critic

$$\phi \leftarrow \phi - \alpha_c \nabla_\phi L(\phi)$$

步骤2：更新Actor

1. 计算策略梯度

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \nabla_a Q_\phi(s_i, a)|_{a=\mu_\theta(s_i)} \cdot \nabla_\theta \mu_\theta(s_i)$$

2. 梯度上升更新Actor

$$\theta \leftarrow \theta + \alpha_a \nabla_\theta J(\theta)$$

步骤3：软更新目标网络

$$\theta' \leftarrow \tau \theta + (1 - \tau) \theta'$$

$$\phi' \leftarrow \tau \phi + (1 - \tau) \phi'$$

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ .

Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q, \theta^{\mu'} \leftarrow \theta^\mu$

Initialize replay buffer R

for episode = 1, M **do**

Initialize a random process $\mathcal{N}$ for action exploration

Receive initial observation state s_1

for t = 1, T **do**

Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise

Execute action a_t and observe reward r_t and observe new state s_{t+1}

Store transition (s_t, a_t, r_t, s_{t+1}) in R

Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R

Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'}))$

Update critic by minimizing the loss: $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$

Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

Update the target networks:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$$

end for
end for

1. 对超参数极度敏感



• 问题表现 & 原因

- 学习率、噪声、 τ 微小变化导致失败，通用性差
- 多组件复杂作用，缺乏信任域约束
- 价值与策略耦合导致误差放大

2. Q值过估计

• 表现 & 原因

- Critic高估导致次优策略，后期性能下降
- 函数近似误差，TD目标最大化偏差
- 贝尔曼迭代误差累积



3. 探索效率问题

• 表现 & 具体问题

- 依赖外部噪声，高维空间效率低
- 固定噪声分布，缺乏状态相关性
- 手动调整探索-利用平衡困难



4. 样本效率与计算效率的权衡

• 表现 & 效率问题

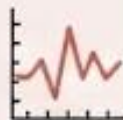
- 需大量交互，四网络更新开销大
- 样本效率低于SAC/TD3
- 目标网络更新慢，批量大小敏感



5. 训练不稳定性

• 表现 & 不稳定来源

- 曲线波动大，易陷局部最优，策略崩溃
- 策略与价值耦合，缺乏策略约束
- 自举误差累积



6. 实现复杂性 (Implementation Complexity)

• 表现 & 复杂性来源

- 需维护四个神经网络，实现经验回放、目标网络、探索噪声
- 调试困难，问题定位复杂
- Critic/Actor/目标网络多步更新，参数同步挑战
- 经验回放内存管理复杂



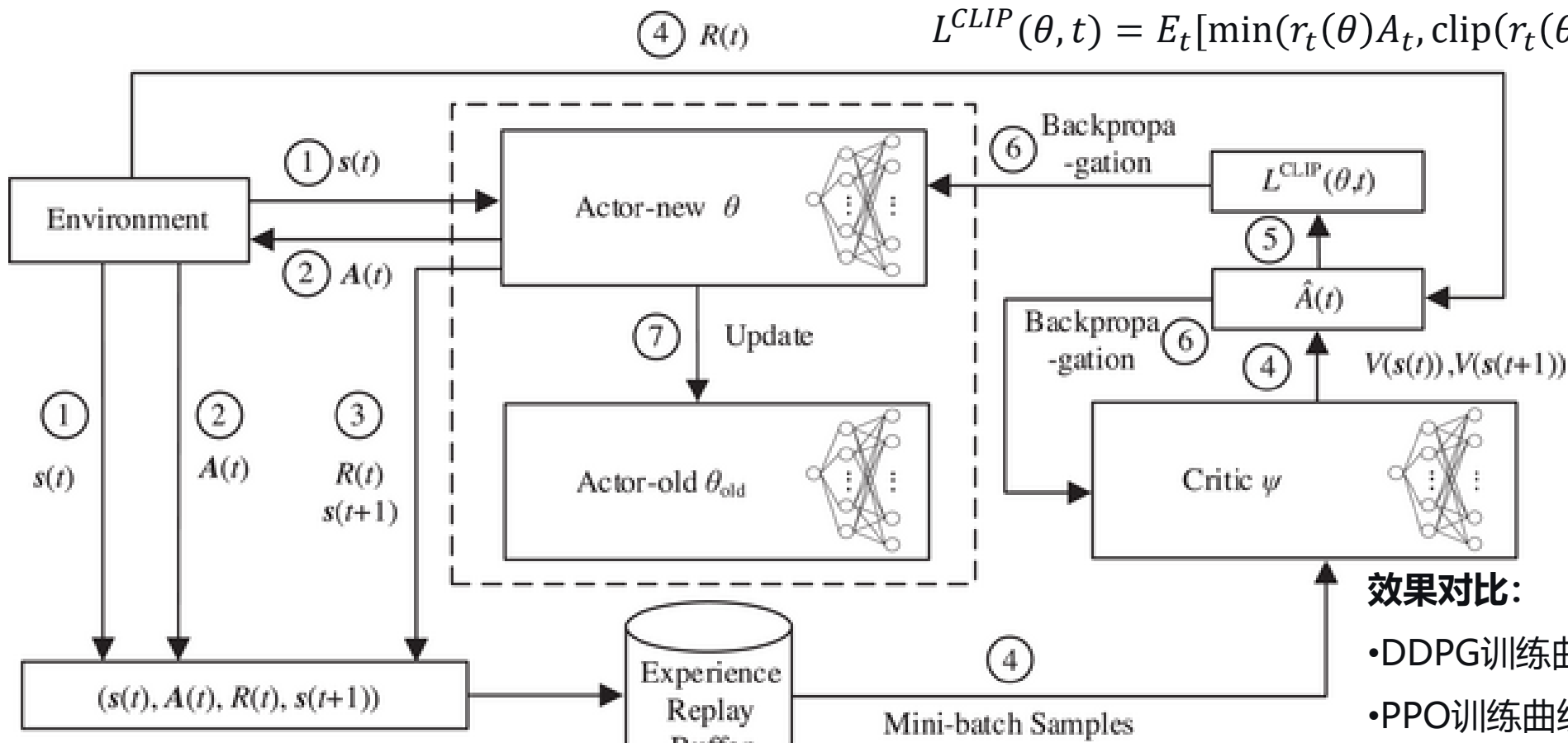


目录

Contents

1. 强化学习基本要素
2. 深度Q神经网络算法DQN
3. 深度确定性策略梯度算法DDPG
- 4. 近端策略优化算法PPO**
5. 领域随机化
6. 实际环境搭建

近端策略优化算法PPO



效果对比:

- DDPG训练曲线: 剧烈波动, 可能突然崩溃
- PPO训练曲线: 平稳上升, 收敛更可靠

PPO的改进:

1. 裁剪机制 (Clipping):

1. 通过 ϵ 参数 (通常0.1-0.3) 硬性限制重要性采样比率
2. 防止策略更新过大, 避免性能突然下降
3. 数学保证: $r_t(\theta) \in [1 - \epsilon, 1 + \epsilon]$

PPO的改进:

2. 信任域思想:

1. 虽然没有显式KL散度约束 (如TRPO)
2. 但裁剪机制隐式约束了策略变化
3. 更简单实现, 避免二阶优化

PPO训练操作流程



中國農業大學
China Agricultural University

阶段0: 初始化 (Initialization)

策略网络 (Policy Network)
 $\pi_{\theta}(a|s) = \mathcal{N}(\mu_{\theta}(s), \sigma_{\theta}^2(s))$

价值网络 (Value Network)
 $V_{\phi}(s)$

优化器 (Optimizer)
Adam($\theta \cup \phi$)

参数共享 (Shared Parameters): $\theta_{\text{shared}} = \phi_{\text{shared}}$

阶段1: 数据收集 (Data Collection) [并行环境 N x 时间步 T]



阶段2: 优势计算 (Advantage Calculation)

1. 计算TD误差 (TD Error)
 $\delta_t = r_t + \gamma(1 - d_t)V(s_{t+1}) - V(s_t)$

2. GAE计算优势 (GAE Advantage)
 $A_t = \sum (\gamma \lambda)^l \delta^{(t+l)}$ (递归高效计算)

3. 计算回报目标 (Return Target)
 $R_t = A_t + V(s_t)$

4. 优势归一化 (Advantage Normalization)
 $\hat{A}_t = \frac{A_t - \mu_A}{\sigma_A + \epsilon}$

阶段3: 策略优化 (Policy Optimization) [K轮更新 K Epochs]

K轮循环

小批量循环

s_t
 a_t
 R_t
 $\hat{A}_t$
 $\log \pi_{\theta}^{\text{old}}$

重要性采样比率 (Ratio) $r_t(\theta) = \pi_{\theta}^{\text{old}}$

裁剪代理损失 (Clip Loss)
 $L_{\text{CLIP}} = \min(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t)$

价值函数损失 (Value Loss)
 $L_{\text{VF}} = 0.5 * (V_{\phi}(s_t) - R_t)^2$

熵奖励 (Entropy Bonus) $S[\pi_{\theta}]$

总损失函数

$L_{\text{PPO}} = -L_{\text{CLIP}} + c_1 L_{\text{VF}} - c_2 S$

梯度更新

$\theta \leftarrow \theta - \alpha \nabla_{\theta} L_{\text{PPO}}$

$\phi \leftarrow \phi - \alpha \nabla_{\phi} L_{\text{PPO}}$

梯度裁剪

对比



维度	DQN (Deep Q-Network)	DDPG (Deep Deterministic Policy Gradient)	PPO (Proximal Policy Optimization)
动作空间	仅离散	仅连续	连续与离散都支持
学习方式	离线策略 (Off-Policy)	离线策略 (Off-Policy)	在线策略 (On-Policy)
核心公式	$y = r + \gamma \max_{a'} Q(s', a')$ $L = (y - Q(s, a))^2$	$\nabla_{\theta} J \approx \mathbb{E}[\nabla_a Q(s, a) \nabla_{\theta} \mu(s)]$ 软更新: $\theta' \leftarrow \tau \theta + (1 - \tau) \theta'$	$L^{\text{CLIP}} = \mathbb{E}[\min(r_t A_t, \text{clip}(r_t, 1 \pm \epsilon) A_t)]$ $\text{GAE: } A_t = \sum (\gamma \lambda)^l \delta_{t+l+1}$
优点	- 概念简单, 易于理解实现 - 离线策略, 样本效率高 - 离散动作任务性能良好 - 可重用历史数据 - 内存使用相对合理	- 直接处理连续动作 - 理论性能上限高 - 推断速度快 (确定性策略) - 离线策略, 可重用数据 - 适合低维连续控制	- 训练稳定, 不易崩溃 - 超参数鲁棒, 调参容易 - 通用性强, 适用多种任务 - 内置智能探索机制 - 实现相对简单
缺点	- 仅支持离散动作 - Q值过估计问题严重 - 探索效率低 (随机ϵ) - 对奖励缩放敏感 - 高维动作空间维度灾难	- 训练极不稳定, 容易发散 - 超参数极度敏感 - 探索机制低效 (需手工设计) - 实现复杂 (4网络维护) - 容易陷入局部最优	- 在线策略, 不能重用旧数据 - 样本效率相对较低 - 需要存储完整轨迹 - 连续动作需要雅可比修正 - 性能不一定最优 (但稳定)
适应场景	- 离散动作游戏 (Atari) - 棋盘游戏 (围棋、象棋) - 简单决策任务 - 教学演示场景	- 连续控制模拟 (MuJoCo) - 物理引擎机器人 - 已知对DDPG友好的特定环境 - 需要确定性策略的应用	- 通用RL任务 (首选基线) - 复杂连续控制 - 生产部署 (稳定性要求高) - 科研实验 (可复现性) - 新手入门项目
不适用场景	- 连续动作任务 - 高维/连续状态+离散动作复杂任务 - 稀疏奖励环境 - 安全关键系统	- 离散动作任务 - 稀疏奖励环境 - 生产系统 (稳定性差) - 多模态最优解任务 - 新手项目	- 必须重用历史数据的场景 - 极端样本效率要求 - 内存严格受限 (需存完整轨迹)

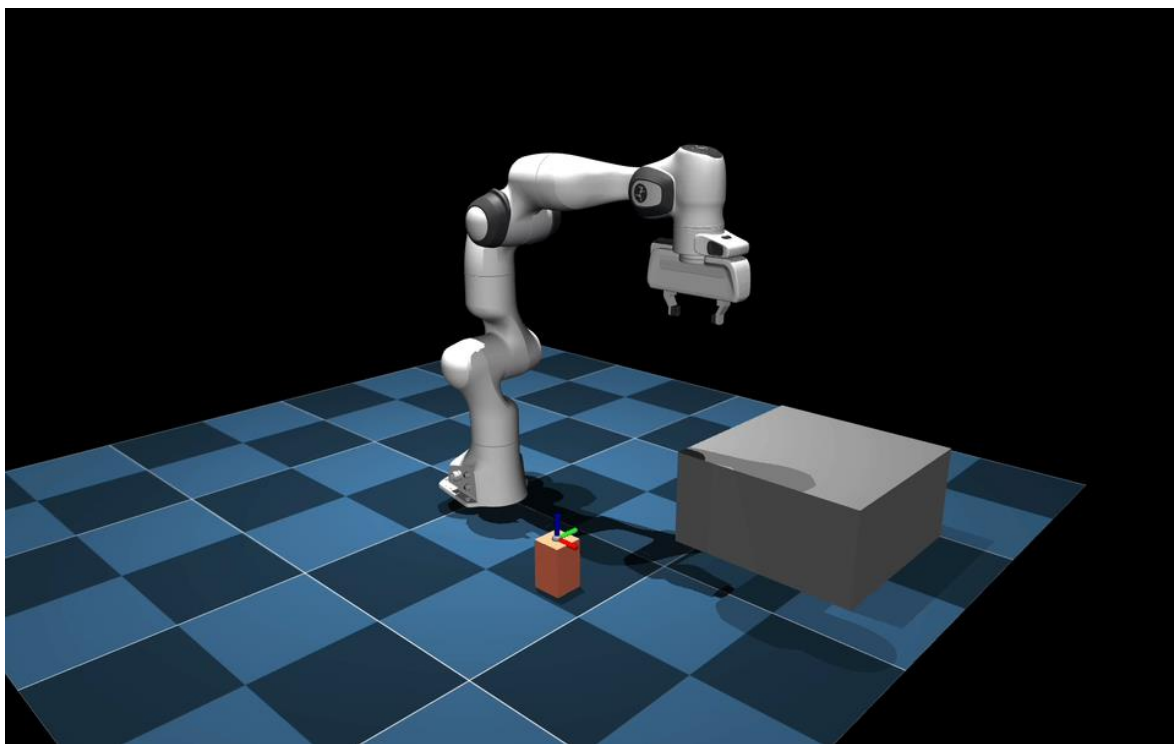


目录

Contents

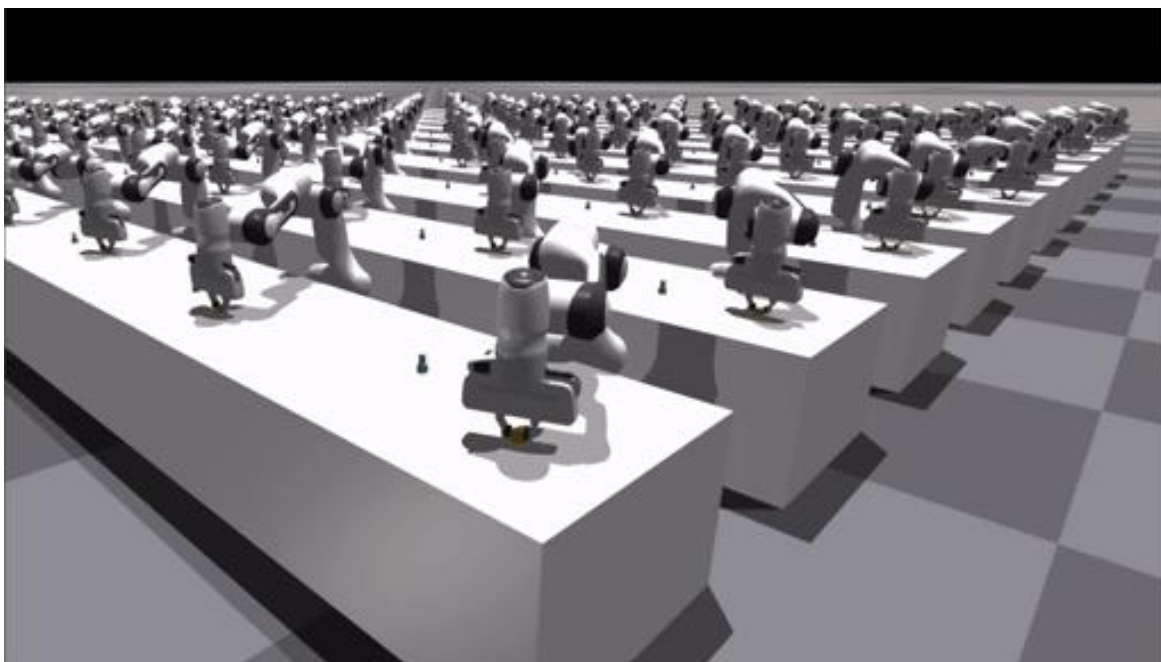
1. 强化学习基本要素
2. 深度Q神经网络算法DQN
3. 深度确定性策略梯度算法DDPG
4. 近端策略优化算法PPO
- 5. 领域随机化**
6. 实际环境搭建

- MuJoCo 的全称是 Multi-Joint dynamics with Contact, 直译为“带接触的多关节动力学”。顾名思义, 它专为高效、准确地模拟包含复杂接触相互作用的机器人或生物体系统而设计。
- MuJoCo 由 Roboti LLC 开发, 其核心开发者是 Emo Todorov (华盛顿大学教授)。2021年10月, DeepMind 收购了 MuJoCo 并将其开源, 这极大地推动了其在学术和工业界的普及。现在, 它由一个由 DeepMind、Roboti 和开源社区共同管理的团队维护。



- 卓越的物理精度与稳定性 (尤其是接触模型)
- 极致的计算性能
- 简洁且强大的建模语言 (MJCF)
- 丰富的内置功能
- 与现代机器学习框架无缝集成

- Isaac Gym 是NVIDIA基于其Omniverse平台构建的、完全GPU加速的机器人仿真与强化学习平台。它的革命性在于：直接在GPU上并行运行成千上万个物理仿真环境，而不是像传统仿真器那样在CPU上串行或少量并行运行。
- Isaac Gym = GPU原生物理仿真 + 大规模并行环境 + 端到端机器学习 workflow



- GPU原生并行仿真（革命性突破）极致的计算性能
- 端到端GPU workflow（无数据拷贝瓶颈）
- 超高的仿真保真度
- 专为强化学习设计的工作流

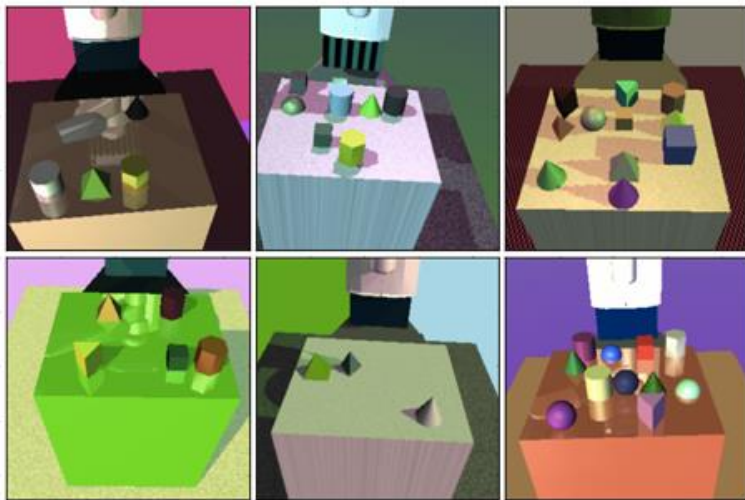
对比维度	MuJoCo	Isaac Gym
核心定位	高精度物理模拟器	大规模并行训练平台
设计理念	“让单个仿真尽可能精确”	“让数千个仿真同时运行”
并行架构	CPU多进程（通常<100个实例）	GPU原生大规模并行 （数千个实例）
物理引擎	专有光滑接触模型（软约束）	PhysX 5（游戏/工业引擎）
训练速度	慢（ 10^4 - 10^5 步/秒）	极快 （ 10^7 - 10^8 步/秒）
物理精度	高（精确摩擦、接触模型）	中等（侧重稳定性）
硬件要求	CPU为主，中等配置即可	必须NVIDIA GPU （高显存）
学习成本	低（文档完善、社区大）	中高（较陡峭）
主要用途	算法研究、控制验证、学术	大规模RL训练、现实部署
成本模型	完全开源免费	研究免费，商用需许可

🎯 一句话选择指南

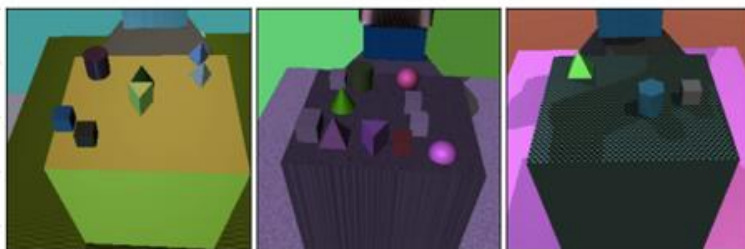
- 选 MuJoCo 如果：你要开发新算法、验证理论、预算有限、需要精确物理。
- 选 Isaac Gym 如果：你要训练部署的策略、有强力GPU、需要快速收敛、做Sim-to-Real。

- 在仿真中训练，在现实中部署。通过域随机化（在仿真中随机化物理参数、视觉外观等）来使策略对真实世界的动态变化具有鲁棒性。(Tobin et al., Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World (IROS 2017))

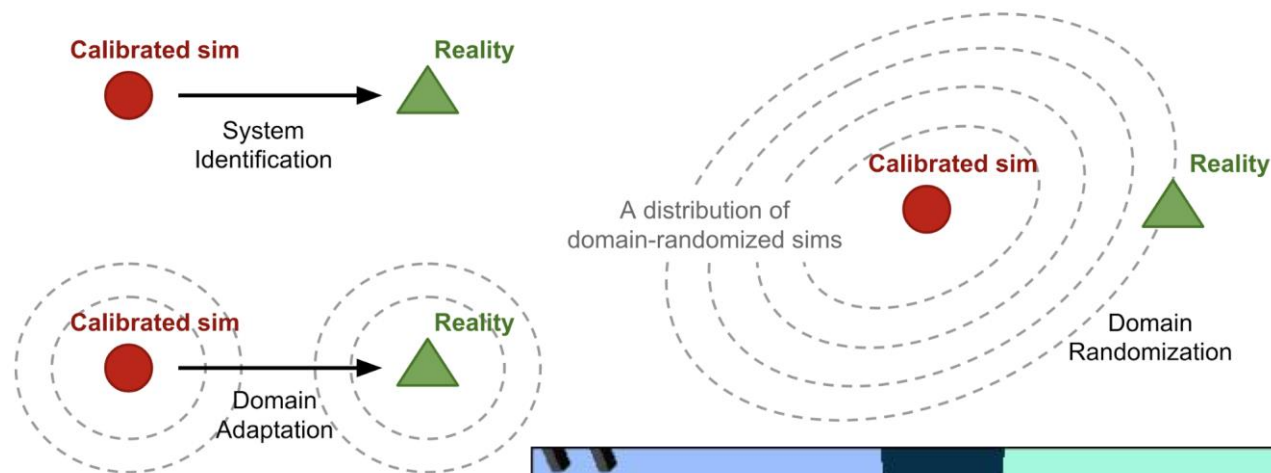
Training



...



Test





目录

Contents

1. 强化学习基本要素
2. 深度Q神经网络算法DQN
3. 深度确定性策略梯度算法DDPG
4. 近端策略优化算法PPO
5. 领域随机化
- 6. 实际环境搭建**

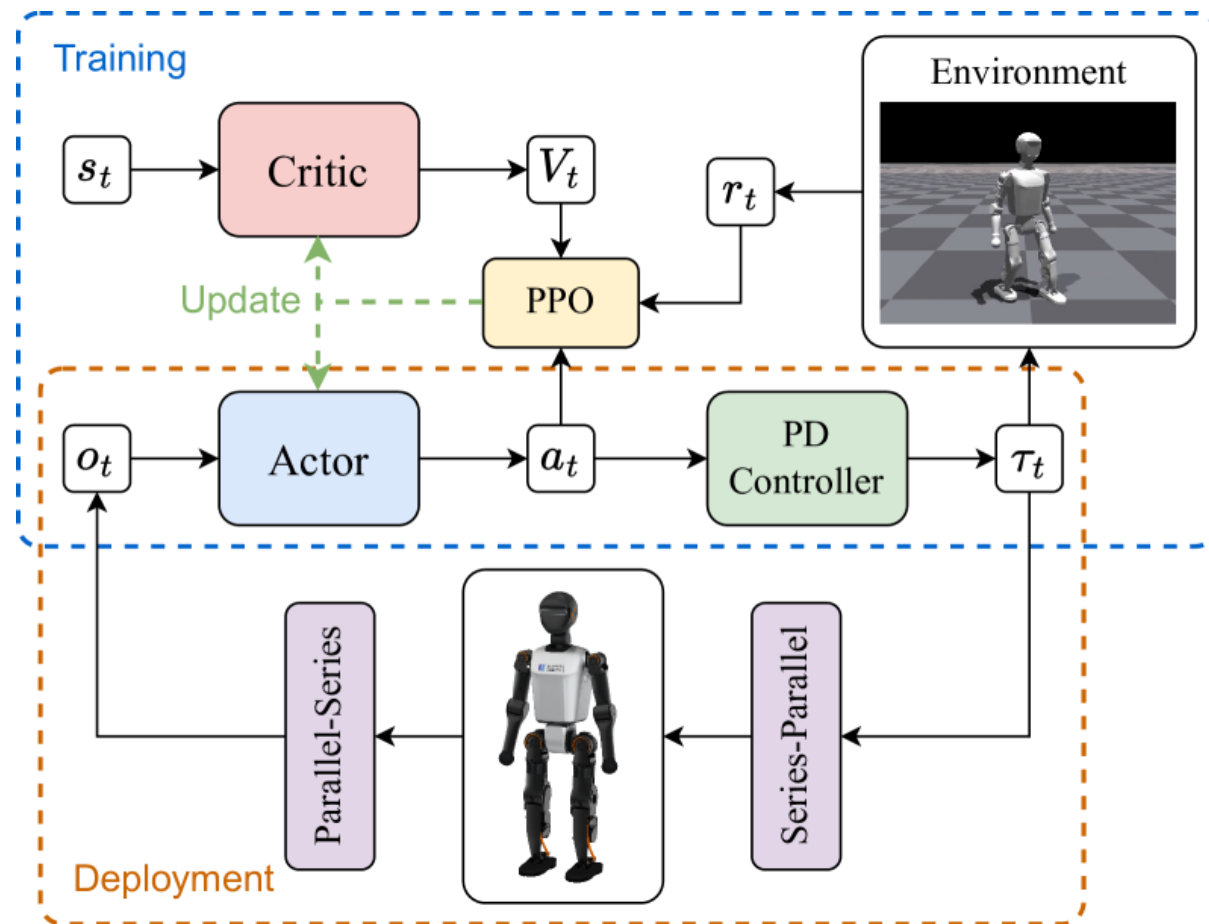
- 机器人控制问题被建模为一个部分可观测马尔可夫决策过程（POMDP），并通过强化学习（RL）进行求解。
- 在该POMDP框架中，智能体仅能获取关于完整状态的部分信息，与现实世界中机器人通过传感器所能获得的信息相一致。
- 智能体的目标是学习一个将观测映射到动作分布的策略，以最大化未来奖励折现总和的期望值。

$$J(\pi) = \mathbb{E}_{\tau \sim p_{\pi}} \left[\sum_{t=0}^{\infty} \gamma^t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

- 具有非对称执行者-评论者（AAC）架构的模型，其中执行者和评论者具有不一致的观测空间，并应用近端策略优化（PPO）作为强化学习算法来训练策略。在每次迭代中，智能体通过执行当前策略收集轨迹，随后使用同批轨迹进行多步优化。策略梯度可表示为：

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim p_{\pi_{\theta_{\text{old}}}}} \left[\frac{\pi_{\theta}(\mathbf{a}_t | \mathbf{o}_t)}{\pi_{\theta_{\text{old}}}(\mathbf{a}_t | \mathbf{o}_t)} A^{\pi_{\theta_{\text{old}}}} \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{o}_t) \right]$$

- 在非对称执行者-评论者架构中，执行者与评论者分别由参数化策略网络 (θ) 和价值网络 (θ) 表示。
- 策略网络根据部分观测信息决策动作，而价值网络通过接收仅在仿真环境中可用的完整状态来估计状态价值。
- 在训练过程中，执行者与评论者网络通过PPO算法在仿真环境中进行协同优化。执行者网络生成的动作经由PD控制器转换为实际控制信号。



- 机器人的本体感知观测包括：基座坐标系中IMU测量的角速度与重力向量，以及通过电机反馈获取的关节位置与速度。我们通过添加噪声的本体感知观测、前一时刻动作以及全向行走任务的速度指令，共同构建执行者的观测空间。
- 此外，观测空间中还引入了步态周期信号($\cos(2\pi ft)$, $\sin(2\pi ft)$)，以引导策略学习周期性步态，其中 f 代表步态频率。当执行站立步态时，步态周期信号设为零，这使得策略能够在站立与行走状态间无缝切换。
- 评论者网络的输入则包含无噪声的执行者观测及其他特权信息，以提升价值估计的准确性。

Components	Dims	Actor	Critic
Commands	3	✓	✓
Gait Cycle	2	✓	✓
Gravity Vector	3	✓	✓
Angular Velocity	3	✓	✓
Joint Position	12	✓	✓
Joint Velocity	12	✓	✓
Previous Action	12	✓	✓
Body Mass	1		✓
Body Center of Mass	3		✓
Base Linear Velocity	3		✓
Base Height	1		✓
Push Force	2		✓
Push Torque	3		✓

- 策略输出关节位置偏移量作为动作向量。机器人的目标关节位置通过以下方式确定：

$$\mathbf{q}_{\text{des}} = \mathbf{q}_0 + \mathbf{a}_t$$

- 其中 $\mathbf{q}_0$ 为默认关节位置。随后，目标关节位置通过电机驱动器的PD控制器直接转换为扭矩指令，该控制器采用固定增益且目标关节速度设为零：

$$\boldsymbol{\tau}_{\text{des}} = \mathbf{k}_p(\mathbf{q}_{\text{des}} - \mathbf{q}) - \mathbf{k}_d\dot{\mathbf{q}}$$

- 策略系统的运行频率为50Hz，而其内置的PD控制器以更高频率运行。相较于直接输出扭矩的控制方式，该设计显著提升了实体机器人上策略执行的稳定性。

基于强化学习的动作训练——奖励函数



- 设计合适的奖励函数对于使机器人获得预期行为至关重要。我们将奖励函数定义为若干奖励分量的加权求和：

$$r(\mathbf{s}_t, \mathbf{a}_t) = \sum w_i r_i$$

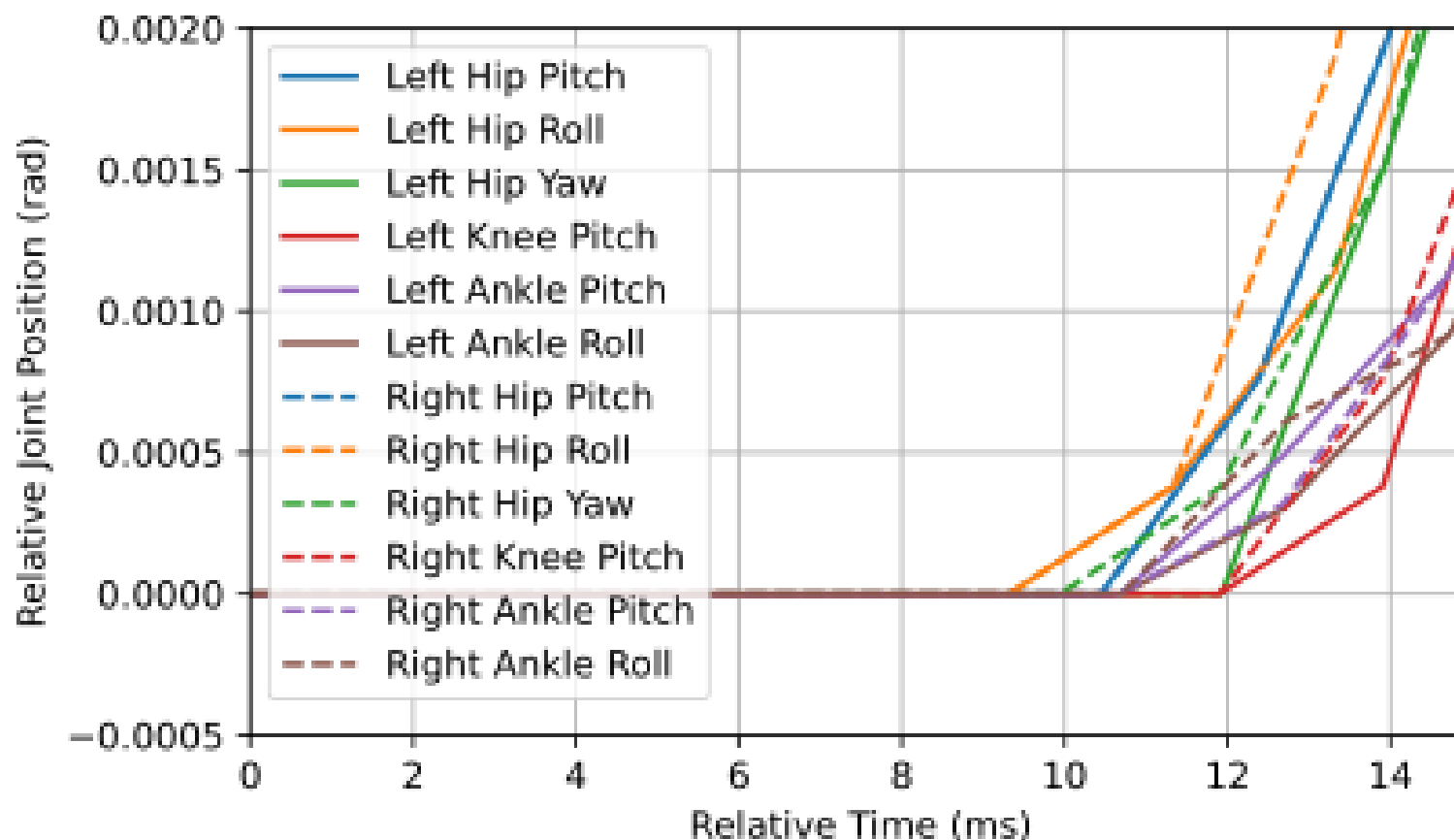
- 奖励函数包含三个组成部分：跟踪奖励、步态奖励和正则化奖励。
- 有关奖励函数设置的具体细分，请参阅表格。

Components	Equations	Weights
Survival	1	0.025
Velocity tracking (x)	$\exp(-(v_x^{\text{cmd}} - v_x)^2 / \sigma_x)$	1.0
Velocity tracking (y)	$\exp(-(v_y^{\text{cmd}} - v_y)^2 / \sigma_y)$	1.0
Velocity tracking (yaw)	$\exp(-(\omega_z^{\text{cmd}} - \omega_z)^2 / \sigma_x)$	0.5
Base height	$(h^{\text{des}} - h)^2$	-20.0
Orientation	$\ \mathbf{g}\ ^2$	-5.0
Torque	$\ \boldsymbol{\tau}\ ^2$	-2×10^{-4}
Torque tiredness	$\ \boldsymbol{\tau} / \boldsymbol{\tau}_{\text{max}}\ ^2$	-1×10^{-2}
Power	$\max(\boldsymbol{\tau} \cdot \dot{\mathbf{q}}, 0)$	-2×10^{-4}
Lin velocity (z)	v_z^2	-2.0
Ang velocity (xy)	$\ \boldsymbol{\omega}_{xy}\ ^2$	-0.2
Joint velocity	$\ \dot{\mathbf{q}}\ ^2$	-1×10^{-4}
Joint acceleration	$\ \ddot{\mathbf{q}}\ ^2$	-1×10^{-7}
Base acceleration	$\ \dot{\mathbf{v}}\ ^2 + \ \dot{\boldsymbol{\omega}}\ ^2$	-1×10^{-4}
Action rate	$\ \mathbf{a}_t - \mathbf{a}_{t-1}\ ^2$	-1.0
Joint position limit	$1_{\mathbf{q} > \mathbf{q}_{\text{max}}} + 1_{\mathbf{q} < \mathbf{q}_{\text{min}}}$	-1.0
Collision	$n_{\text{collision}}$	-1.0
Feet swing	$1_{\text{feet swing}} \cdot 1_{\text{swing period}}$	3.0
Feet slip	$1_{\text{feet stance}} \cdot \ \mathbf{v}_{\text{feet}}\ ^2$	-0.1
Feet yaw	$\ \boldsymbol{\psi}_{\text{feet}} - \boldsymbol{\psi}_{\text{base}}\ ^2$	-1.0
Feet roll	$\ \boldsymbol{\phi}_{\text{feet}}\ ^2$	-0.1
Feet distance	$\max(d_{\text{ref}} - d_{\text{feet}}, 0)$	-1.0

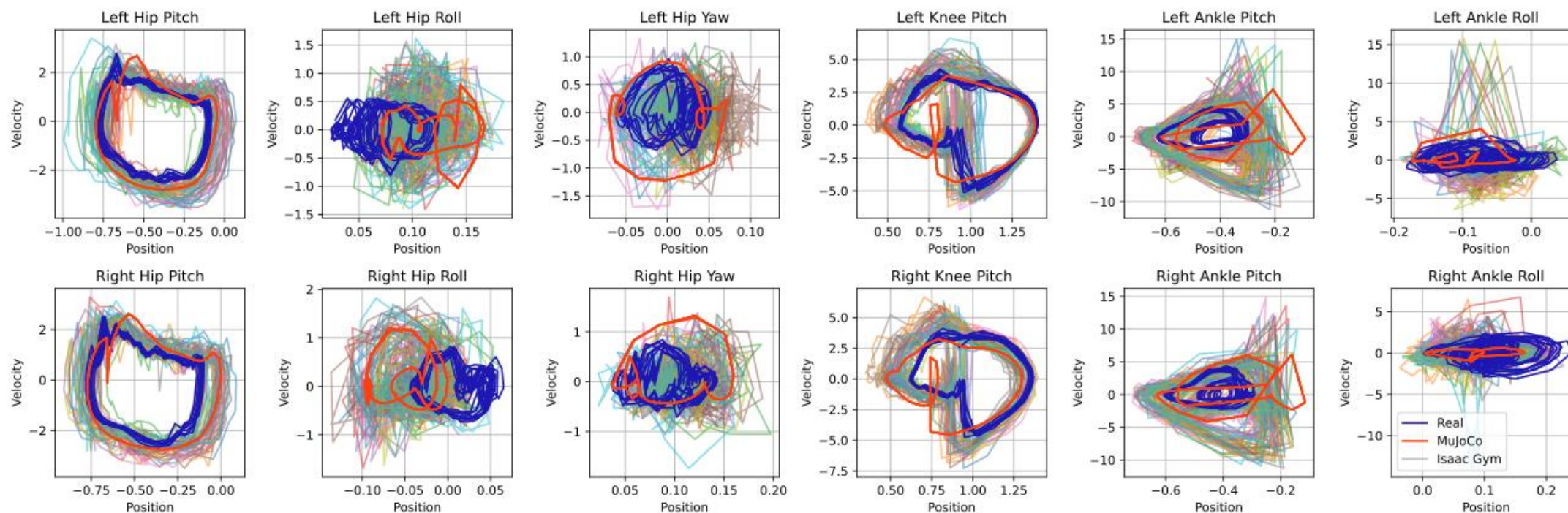
- 每个训练回合的最大持续时间设置为1500步，相当于30秒。当超过时间限制时，环境会重置，但后续奖励仍将计入广义优势估计（GAE）。
- 为防止智能体在无效状态（如机器人跌倒时）浪费时间探索，设置了额外的提前终止条件，包括基座高度过低或基座速度超过阈值。在这些条件下，环境将立即重置且不授予后续奖励，从而防止机器人摔倒或出现极端行为。
- 此外，将每帧的总奖励限制为零以上，以避免智能体因获得负奖励而倾向于提前终止回合。

- 由于现实世界数据收集在效率和安全方面存在局限，强化学习（RL）通常会在仿真环境中训练策略，并以零样本迁移的方式将其部署到现实世界。然而，仿真环境中的理想化假设会引发仿真到现实的差异问题，这些假设未能充分考虑现实世界中的不确定性因素，例如传感器噪声、执行器动态特性和环境变化。
- 为应对仿真到现实的差异，采用领域随机化方法来弥合仿真环境与现实部署之间的差距。针对仿人机器人运动问题，这些不确定性被归纳为三个主要来源：机器人本体动力学、执行器特性以及环境条件。通过在仿真训练中对这些类别分别实施领域随机化，使策略能够适应各种现实场景。

- 为提升仿真与现实的契合度，对领域随机化所需的关键现实世界参数进行了定量标定。
例如，通过机器人在板CPU的Python SDK发送关节指令并读取关节位置反馈，测得从指令发送到首次观测到关节位置变化的往返通信延迟为9-12毫秒。



- 通过对比Isaac Gym中的训练结果与现实场景中的部署表现，验证精心设计的领域随机化能有效覆盖现实世界特性，成功弥合仿真与现实间的差异，实现策略向实体机器人的零样本迁移。此外，基于MuJoCo的跨仿真测试结果表明，该环境可作为高度契合现实动力学的有效验证平台。



- 为便于将训练好的策略部署到实体机器人，基于Booster Robotics SDK开发了Python部署框架，并配套提供预训练策略，能够在真实硬件上无缝实施和评估强化学习策略。
- 针对并联踝关节机构中闭链运动学的处理难题，在SDK中实现了串并联转换模块。该模块支持用户使用虚拟串联结构无缝训练策略，同时无需增加额外复杂度即可在并联机构上进行部署。



全向运动



中國農業大學
China Agricultural University



全进后退

全向运动



中國農業大學
China Agricultural University

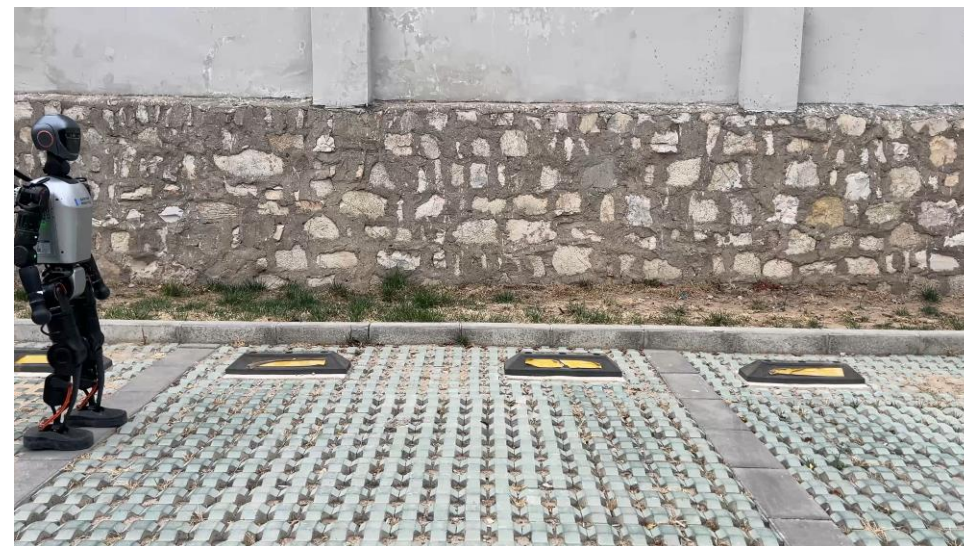


侧向

崎岖路面



中國農業大學
China Agricultural University



崎岖路面



中國農業大學
China Agricultural University





前向和后向行走



侧向行走

保持平衡



中國農業大學
China Agricultural University



10kg 推挤



中國農業大學
China Agricultural University

强化学习展新篇，智能交互探自然。
策略评估深度算，奖励回馈引智前。

